

Phương pháp chia để trị (Divide and Conquer)

Giảng viên: Trần Đình Tân

Môn: Thuật toán ứng dụng

Sau bài học, người học có thể:

- Phát biểu ba bước chia để trị (Divide, Conquer, Combine) và viết phương trình truy hồi tương ứng
- Áp dụng phương pháp cây đệ quy và định lý thợ để phân tích hệ thức truy hồi
- Thiết kế và phân tích thuật toán Tìm kiếm nhị phân, Merge Sort, Quick Sort theo khuôn khổ chia để trị
- Phân biệt khi nào chia để trị hiệu quả và nhận ra giới hạn của phương pháp

Nội dung

- 1 Ý tưởng và nguyên lý cốt lõi
- 2 Phân tích hệ thức truy hồi
- 3 Tìm kiếm nhị phân
- 4 Sắp xếp trộn (Merge Sort)
- 5 Sắp xếp nhanh (Quick Sort)
- 6 Phân tích và ứng dụng
- 7 Tổng kết

Chia để trị là gì?

Định nghĩa

Thuật toán **chia để trị** (divide and conquer) giải bài toán bằng cách đệ quy phân rã thành các bài toán con *cùng cấu trúc*, giải từng bài toán con, rồi kết hợp kết quả lại.

Trực giác: Bài toán lớn và khó thường dễ hơn khi chia thành các phần nhỏ hơn cùng loại.

Ví dụ trong cuộc sống:

- Tìm trong từ điển: mở ở giữa, quyết định tìm nửa nào
- Chia công việc lớn cho nhiều nhóm nhỏ, tổng hợp kết quả
- Cân hàng hóa bằng cân đĩa: chia đôi lô để tìm lô nặng hơn

Điểm khác biệt then chốt: Các bài toán con **không chồng lấp** nhau (phân biệt với quy hoạch động).

Ba bước của khuôn khổ chia để trị

1 **Divide** — Chia:

Chia bài toán kích thước n thành $a \geq 1$ bài toán con, mỗi bài có kích thước n/b ($b > 1$).

2 **Conquer** — Trị:

Giải đệ quy từng bài toán con. Khi đủ nhỏ (trường hợp cơ sở), giải trực tiếp trong $\Theta(1)$.

3 **Combine** — Kết hợp:

Ghép nghiệm các bài toán con thành nghiệm bài toán gốc.

Một số thuật toán không cần bước này.

Phương trình truy hồi tổng quát

$$T(n) = a T\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, b > 1$$

$f(n)$ = chi phí tổng hợp của bước Divide và Combine.

Quy trình thiết kế thuật toán chia để trị

- 1 **Xác định** bài toán con: cùng loại, kích thước nhỏ hơn
- 2 **Chọn** cách phân chia: chia đôi, chia k phần, ...
- 3 **Xác định** trường hợp cơ sở: giải trực tiếp khi $n \leq c$
- 4 **Thiết kế** bước Combine sao cho chi phí hợp lý
- 5 **Viết** phương trình truy hồi $T(n) = aT(n/b) + f(n)$
- 6 **Giải** $T(n)$ bằng phương pháp thế, cây đệ quy hoặc định lý thore

Ba bài toán kinh điển trong bài này

Thuật toán	a	b	Hệ thức truy hồi
Tìm kiếm nhị phân	1	2	$T(n) = T(n/2) + \Theta(1)$
Merge Sort	2	2	$T(n) = 2T(n/2) + \Theta(n)$
Quick Sort	2	≈ 2 (tb)	$T(n) \approx 2T(n/2) + \Theta(n)$

Chia để trị, Tham lam và Quy hoạch động

Chia để trị

- Bài toán con *không* chồng lấp
- Giải đệ quy độc lập
- Kết hợp kết quả
- VD: Merge Sort, Binary Search

Quy hoạch động

- Bài toán con *chồng lấp*
- Lưu bảng, tránh tính lại
- Tổng hợp từ bảng
- VD: Fibonacci, ba lô 0-1

Tham lam

- Chọn tốt nhất cục bộ
- Không xét lại
- Không cần bài toán con
- VD: Huffman, Activity Selection

Khi bài toán con chồng lấp

Chia để trị sẽ tính lại nhiều lần $\Rightarrow$ hiệu năng kém.

Ví dụ: Fibonacci bằng đệ quy tốn $\Theta(2^n)$; quy hoạch động chỉ tốn $\Theta(n)$.

Giới thiệu ba phương pháp phân tích

Cho hệ thức $T(n) = aT(n/b) + f(n)$. Ba công cụ chính:

❶ **Phương pháp thế (Substitution method):**

Đoán dạng $T(n)$, dùng quy nạp toán học để chứng minh.

Ưu: linh hoạt. *Nhược:* cần đoán đúng dạng trước.

❷ **Phương pháp cây đệ quy (Recursion tree):**

Vẽ cây khai triển đệ quy, tính tổng chi phí từng tầng.

Ưu: trực quan, gợi ý dạng cho phương pháp thế.

❸ **Định lý thợ (Master Theorem):**

Công thức đóng trực tiếp cho $T(n) = aT(n/b) + f(n)$.

Ưu: nhanh, không cần tính toán phức tạp. *Nhược:* không áp dụng mọi trường hợp.

Phương pháp cây đệ quy — Ví dụ Merge Sort

Hệ thức: $T(n) = 2T(n/2) + cn, c > 0$.

Chi phí từng tầng:

- Tầng 0: 1 nút $\rightarrow cn$
- Tầng 1: 2 nút $\cdot cn/2 = cn$
- Tầng 2: 4 nút $\cdot cn/4 = cn$
- ... (mỗi tầng đều cn)
- Tầng k : 2^k nút $\cdot cn/2^k = cn$
- Tầng $\log_2 n$: n lá $\cdot \Theta(1) = \Theta(n)$

Tổng chi phí:

Số tầng không lá $= \log_2 n$

Chi phí mỗi tầng không lá $= cn$

$$T(n) = cn \log_2 n + \Theta(n) = \Theta(n \log n)$$

Nhận xét: Chi phí đều nhau ở mỗi tầng là dấu hiệu điển hình của Trường hợp 2 trong Định lý thợ.

Định lý thợ (Master Theorem)

Định lý (CLRS Chương 4)

Cho $T(n) = aT(n/b) + f(n)$, $a \geq 1$, $b > 1$, f tiệm cận dương. Đặt $n^{\log_b a}$ là hàm trọng tâm. Ba trường hợp:

- 1 Nếu $f(n) = O(n^{\log_b a - \varepsilon})$ với $\varepsilon > 0 \Rightarrow T(n) = \Theta(n^{\log_b a})$
- 2 Nếu $f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$
- 3 Nếu $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, và $af(n/b) \leq cf(n)$ với $c < 1 \Rightarrow T(n) = \Theta(f(n))$

Trực giác: So sánh $f(n)$ với hàm trọng tâm $n^{\log_b a}$.

Cái nào “nặng” hơn đa thức sẽ chiếm ưu thế; nếu cân bằng thêm $\log n$.

Áp dụng Định lý thặng

Hệ thức	a	b	$n^{\log_b a}$	TH	Kết quả
$T(n) = 2T(n/2) + \Theta(n)$	2	2	n	2	$\Theta(n \log n)$
$T(n) = T(n/2) + \Theta(1)$	1	2	1	2	$\Theta(\log n)$
$T(n) = 4T(n/2) + \Theta(n)$	4	2	n^2	1	$\Theta(n^2)$
$T(n) = 4T(n/2) + \Theta(n^3)$	4	2	n^2	3	$\Theta(n^3)$
$T(n) = 2T(n/2) + \Theta(n^2)$	2	2	n	3	$\Theta(n^2)$

Khi Định lý thặng không áp dụng được

$T(n) = 2T(n/2) + n/\log n$: $f(n) = n/\log n$ không thoả $O(n^{1-\epsilon})$ với bất kỳ $\epsilon > 0$ nào. Cần dùng phương pháp cây đệ quy.

Bài toán và ý tưởng tìm kiếm nhị phân

Bài toán

Cho mảng $A[1..n]$ đã sắp xếp tăng dần và khóa k . Tìm chỉ số i sao cho $A[i] = k$; trả về -1 nếu không tồn tại.

Khuôn khổ chia để trị:

- **Divide:** Xét phần tử giữa $m = \lfloor (p + r)/2 \rfloor$; so sánh k với $A[m]$.
- **Conquer:** Tìm đệ quy trong nửa phù hợp (kích thước $\approx n/2$).
- **Combine:** Không cần — kết quả trả trực tiếp từ bài toán con.

Ví dụ: $A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]$, $k = 23$:

Bước	Phạm vi $[p..r]$	$A[m]$	Quyết định
1	$[1..10]$, $m = 5$	$16 < 23$	Tìm bên phải $[6..10]$
2	$[6..10]$, $m = 8$	$56 > 23$	Tìm bên trái $[6..7]$
3	$[6..7]$, $m = 6$	$23 = k$	Tìm thấy tại $i = 6$ ✓

Thuật toán và phân tích tìm kiếm nhị phân

Algorithm 1 BINARY-SEARCH(A, p, r, k)

```
1: if  $p > r$  then
2:   return  $-1$ 
3: end if
4:  $m \leftarrow \lfloor (p + r) / 2 \rfloor$ 
5: if  $k = A[m]$  then
6:   return  $m$ 
7: else if  $k < A[m]$  then
8:   return BINARY-SEARCH( $A, p, m - 1, k$ )
9: else
10:  return BINARY-SEARCH( $A, m + 1, r, k$ )
11: end if
```

Phân tích: $T(n) = T(n/2) + \Theta(1)$.

Bài toán và ý tưởng Merge Sort

Bài toán (CLRS 2.3)

Sắp xếp mảng $A[p..r]$ theo thứ tự tăng dần.

Khuôn khổ chia để trị:

- **Divide:** $A[p..r] \rightarrow A[p..q]$ và $A[q + 1..r]$, $q = \lfloor (p + r)/2 \rfloor$. Chi phí $\Theta(1)$.
- **Conquer:** Đệ quy Merge Sort trên hai nửa. Chi phí $2T(n/2)$.
- **Combine:** Trộn hai nửa đã sắp xếp (MERGE). Chi phí $\Theta(n)$.

Trường hợp cơ sở: $p \geq r$ — không làm gì.

Phương trình truy hồi:

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 2T(n/2) + \Theta(n) & n > 1 \end{cases} \xrightarrow{\text{Định lý thặng TH 2}} T(n) = \Theta(n \log n)$$

Thuật toán MERGE-SORT và cây đệ quy

Algorithm 2 MERGE-SORT(A, p, r)

```
1: if  $p < r$  then  
2:    $q \leftarrow \lfloor (p + r)/2 \rfloor$   
3:   MERGE-SORT( $A, p, q$ )  
4:   MERGE-SORT( $A, q+1, r$ )  
5:   MERGE( $A, p, q, r$ )  
6: end if
```

$A = [5, 2, 4, 6, 1, 3, 2, 6], n = 8:$

Chia	$[5, 2, 4, 6] \mid [1, 3, 2, 6]$
	$[5, 2] \mid [4, 6] \mid [1, 3] \mid [2, 6]$
	8 mảng 1 phần tử

Merge	$[2, 5] \ [4, 6] \ [1, 3] \ [2, 6]$
	$[2, 4, 5, 6] \ [1, 2, 3, 6]$
	$[1, 2, 2, 3, 4, 5, 6, 6]$

Tổng chi phí: $\log_2 8 = 3$ tầng,
mỗi tầng merge tốn $\Theta(n)$.

Thuật toán MERGE

Algorithm 3 MERGE(A, p, q, r)

```
1:  $n_1 \leftarrow q - p + 1$ ;  $n_2 \leftarrow r - q$ 
2: for  $i = 1$  to  $n_1$  do
3:    $L[i] \leftarrow A[p + i - 1]$ 
4: end for
5: for  $j = 1$  to  $n_2$  do
6:    $R[j] \leftarrow A[q + j]$ 
7: end for
8:  $L[n_1 + 1] \leftarrow \infty$ ;  $R[n_2 + 1] \leftarrow \infty$ ;  $i \leftarrow 1$ ;  $j \leftarrow 1$ 
9: for  $k = p$  to  $r$  do
10:  if  $L[i] \leq R[j]$  then
11:     $A[k] \leftarrow L[i]$ ;  $i \leftarrow i + 1$ 
12:  else
13:     $A[k] \leftarrow R[j]$ ;  $j \leftarrow j + 1$ 
14:  end if
```

Minh họa MERGE: trộn $[2, 4, 5]$ và $[1, 3, 6]$

$L = [2, 4, 5, \infty]$, $R = [1, 3, 6, \infty] \rightarrow A[1..6]$:

Bước k	$L[i]$	$R[j]$	Chọn	$A[1..k]$
1	2	1	$R[1] = 1$	[1]
2	2	3	$L[1] = 2$	[1, 2]
3	4	3	$R[2] = 3$	[1, 2, 3]
4	4	6	$L[2] = 4$	[1, 2, 3, 4]
5	5	6	$L[3] = 5$	[1, 2, 3, 4, 5]
6	∞	6	$R[3] = 6$	[1, 2, 3, 4, 5, 6]

Tổng số phép so sánh: $\leq n_1 + n_2 = n$. Độ phức tạp MERGE: $\Theta(n)$.

Bài toán và ý tưởng Quick Sort

Bài toán (CLRS 7)

Sắp xếp mảng $A[p..r]$ tại chỗ (in-place) theo thứ tự tăng dần.

Khuôn khổ chia để trị:

- **Divide:** Phân hoạch $A[p..r]$ quanh chốt (pivot) $x = A[r]$ thành $A[p..q-1] \leq x$ và $A[q+1..r] \geq x$. Chi phí $\Theta(n)$.
- **Conquer:** Đệ quy Quick Sort trên $A[p..q-1]$ và $A[q+1..r]$.
- **Combine:** Không cần — các dãy con đã ở đúng vị trí.

So sánh với Merge Sort

	Merge Sort	Quick Sort
Divide	$\Theta(1)$, chia đều	$\Theta(n)$, chia tùy chốt
Combine	$\Theta(n)$ (trộn)	Không cần
Bộ nhớ	$O(n)$ phụ	$O(1)$ (in-place)

Algorithm 4 PARTITION(A, p, r)

```
1:  $x \leftarrow A[r]$ 
2:  $i \leftarrow p - 1$ 
3: for  $j = p$  to  $r - 1$  do
4:   if  $A[j] \leq x$  then
5:      $i \leftarrow i + 1$ 
6:     swap( $A[i], A[j]$ )
7:   end if
8: end for
9: swap( $A[i + 1], A[r]$ )
10: return  $i + 1$ 
```

Bất biến vòng lặp:

Tại mỗi lần lặp j :

- $A[p..i] \leq x$
- $A[i + 1..j - 1] > x$
- $A[r] = x$ (chốt ở cuối)

Kết thúc:

$A[p..q-1] \leq x = A[q] \leq A[q+1..r]$
 $\Rightarrow$ chốt ở đúng vị trí cuối cùng.

Thuật toán QUICK-SORT và minh họa PARTITION

Algorithm 5 QUICK-SORT(A, p, r)

```
1: if  $p < r$  then  
2:    $q \leftarrow \text{PARTITION}(A, p, r)$   
3:   QUICK-SORT( $A, p, q-1$ )  
4:   QUICK-SORT( $A, q+1, r$ )  
5: end if
```

PARTITION trên $[2, 8, 7, 1, 3, 5, 6, 4]$, $x = 4$:

j	$A[j]$	So sánh	Hành động
1	2	≤ 4	$i = 1$, swap $A[1], A[1]$
2	8	> 4	bỏ qua
3	7	> 4	bỏ qua
4	1	≤ 4	$i = 2$, swap $A[2], A[4]$
5	3	≤ 4	$i = 3$, swap $A[3], A[5]$
6	5	> 4	bỏ qua
7	6	> 4	bỏ qua

swap $A[4], A[8]$: $[2, 1, 3, 4, 8, 5, 6, 7]$

Chốt đặt tại $q = 4$

Phân tích độ phức tạp Quick Sort

Trường hợp xấu nhất: Phân hoạch lệch tối đa ($q = p$ hoặc $q = r$); xảy ra khi dãy đã sắp xếp và chọn chốt đầu/cuối.

$$T(n) = T(0) + T(n-1) + \Theta(n) = T(n-1) + \Theta(n) \Rightarrow T(n) = \Theta(n^2)$$

Trường hợp tốt nhất: Phân hoạch đều hai nửa.

$$T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$$

Trường hợp trung bình: Phân hoạch tỉ lệ 9 : 1 (lệch hẳn):

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) = O(n \log n)$$

Kết luận

Tốt nhất và trung bình: $O(n \log n)$. Xấu nhất: $\Theta(n^2)$ — hiếm gặp trong thực tế với dữ liệu ngẫu nhiên.

Quick Sort ngẫu nhiên hóa

Vấn đề với chốt cố định: Kẻ tấn công có thể tạo đầu vào khiến Quick Sort luôn đạt $\Theta(n^2)$.

Giải pháp: Chọn chốt **ngẫu nhiên** từ $A[p..r]$:

Algorithm 6 RANDOMIZED-PARTITION(A, p, r)

- 1: $i \leftarrow \text{RANDOM}(p, r)$
 - 2: $\text{swap}(A[i], A[r])$
 - 3: **return** PARTITION(A, p, r)
-

Kết quả (CLRS 7.4): $\mathbb{E}[T(n)] = O(n \log n)$ với *mọi* đầu vào, không phụ thuộc phân phối dữ liệu.

Tại sao Quick Sort thường nhanh hơn Merge Sort trong thực tế?

- Hằng số ẩn trong $O(n \log n)$ nhỏ hơn
- Sắp xếp tại chỗ: thân thiện với bộ nhớ đệm (cache-friendly)
- Không cần cấp phát mảng phụ $O(n)$

So sánh các thuật toán kinh điển

Thuật toán	Tốt nhất	Trung bình	Xấu nhất	Bộ nhớ phụ	Ổn định
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	—
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Có
Quick Sort	$O(n \log n)$	$O(n \log n)$	$\Theta(n^2)$	$O(\log n)$	Không

Nhận xét thực tế:

- **Merge Sort:** Đảm bảo $\Theta(n \log n)$; dùng khi cần ổn định (sắp xếp nhiều tiêu chí) hoặc sắp xếp ngoài bộ nhớ (external sort)
- **Quick Sort ngẫu nhiên hóa:** $O(n \log n)$ kỳ vọng, hằng số nhỏ, cache-friendly $\Rightarrow$ thường là lựa chọn thực tế
- **Thư viện:** `std::sort` (C++) dùng Introsort (Quick + Heap + Insertion) để đảm bảo $O(n \log n)$ mọi trường hợp

Ứng dụng chia để trị trong thực tế

Ứng dụng trực tiếp

- Sắp xếp: Merge Sort, Quick Sort, Introsort
- Tìm kiếm: Binary Search, B-tree
- Nhân ma trận Strassen: $O(n^{2.81})$ thay vì $O(n^3)$
- Cập điểm gần nhất: $O(n \log n)$ thay vì $O(n^2)$
- Thuật toán Karatsuba (nhân số nguyên lớn): $O(n^{1.58})$

Quy tắc kinh nghiệm: Chia để trị mang lại cải thiện khi bước Combine tốn $O(n)$ hoặc ít hơn và số bài toán con a đủ nhỏ.

Ứng dụng gián tiếp

- **MapReduce:** chia dữ liệu lớn, xử lý song song, tổng hợp
- **FFT:** biến đổi Fourier nhanh $O(n \log n)$, nền tảng xử lý tín hiệu và nén ảnh
- **Cây tìm kiếm cân bằng:** AVL, Red-Black
- **Tính toán song song:** phân bài toán cho nhiều bộ xử lý

- **Khuôn khổ:** Chia để trị gồm ba bước Divide – Conquer – Combine; phương trình truy hồi $T(n) = aT(n/b) + f(n)$.
- **Phân tích:** Ba công cụ — phương pháp thế, cây đệ quy, và định lý thợ. Định lý thợ nhanh nhất cho dạng $aT(n/b) + f(n)$.
- **Tìm kiếm nhị phân:** Không cần Combine; $T(n) = \Theta(\log n)$.
- **Merge Sort:** Combine tốn $\Theta(n)$; $T(n) = \Theta(n \log n)$ mọi trường hợp; ổn định; không in-place.
- **Quick Sort:** Divide tốn $\Theta(n)$; xấu nhất $\Theta(n^2)$ nhưng trung bình $O(n \log n)$; in-place; ngẫu nhiên hóa loại bỏ đầu vào cổ tình xấu.
- **Giới hạn:** Kém hiệu quả khi bài toán con chồng lấp (dùng quy hoạch động thay thế).

- 1 Cho tập S gồm n số nguyên và số nguyên x . Kiểm tra xem có tồn tại $a, b \in S$ sao cho $a + b = x$ không. Đề xuất thuật toán $O(n^2)$; sau đó cải thiện lên $O(n \log n)$.
- 2 Sắp xếp $A = [3, 1, 4, 1, 5, 9, 2, 6]$ bằng Merge Sort. Vẽ cây đệ quy đầy đủ và đếm số phép so sánh.
- 3 Chạy PARTITION trên $A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$, chốt $x = A[12] = 11$. Cho biết vị trí chốt sau khi phân hoạch.
- 4 Giải hệ thức $T(n) = 3T(n/4) + \Theta(n \log n)$ bằng định lý thợ. Gợi ý: tính $n^{\log_4 3}$ và so sánh với $f(n)$.
- 5 **(Nâng cao)** Thuật toán Karatsuba nhân hai số n chữ số có hệ thức $T(n) = 3T(n/2) + \Theta(n)$. Tính độ phức tạp và so sánh với nhân thông thường $O(n^2)$.

- T.H. Cormen et al., *Introduction to Algorithms*, 4th ed., MIT Press, 2022 — Chương 2.3 (Merge Sort), 4 (Divide and Conquer, Master Theorem), 7 (Quick Sort).
- J. Kleinberg, É. Tardos, *Algorithm Design*, Addison-Wesley, 2005 — Chương 5 (Divide and Conquer).
- S. Dasgupta, C. Papadimitriou, U. Vazirani, *Algorithms*, McGraw-Hill, 2006 — Chương 2 (Divide and Conquer).
- Bài giảng tham khảo: *Phương pháp chia để trị*, Tài liệu môn học, Chương 3.